# CUESTIONARIO 9

## Excepciones

**Pregunta 1**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

In Java, which class is the superclass of all errors and exceptions?

Seleccione una:

a. Object
b. RuntimeException
c. Exception
d. Error
e. Throwable ✓

**Justificación:** Throwable es la raíz de la jerarquía de excepciones, y de ella heredan Error y Exception. Object es la raíz de todas las clases, no específicamente de los errores y las excepciones. Exception y RuntimeException son subclases.

Fuente: Java.pdf #146

La respuesta correcta es: Throwable

| **Mi justificación** |
| --- |
| Todos los errores y excepciones son hijos de throwable |

**Pregunta 2**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuáles son los tipos de excepciones en Java?

Seleccione una o más de una:

- a. Throwable
- b. Object
- c. Checked ✓
- d. Unchecked ✓
- e. Todas las opciones son correctas.

**Justificación:** Técnicamente, las excepciones en Java se clasifican en checked (el compilador exige capturarlas o declararlas) y unchecked (RuntimeException y sus subclases, además de Error); esas son la respuesta correcta (c y d). Throwable es la superclase raíz de toda la jerarquía, no un tipo de excepción, pero el material de Accenture (Java.pdf, misma pregunta) la marca también como correcta; por eso aquí Throwable es neutra: no suma ni resta. Object ni siquiera es una excepción, así que 'Todas' tampoco es correcta.

*Nota sobre el material original:* El material de Accenture (Java.pdf) da como correctas Throwable, Checked y Unchecked. Técnicamente solo Checked y Unchecked son tipos; aquí marcar Throwable no suma ni resta.

Fuente: Simulación APX V1 #13 · también aparece en: Java.pdf #1, Java.pdf #180

Las respuestas correctas son: Checked, Unchecked

| **Mi justificación** |
| --- |
| Las excepciones se clasifican en Checked y Unchecked. En las primeras se deben capturar con catch para decidir qué hacer en caso de que ocurran, y en las segundas no. |

**Pregunta 3**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuáles de las siguientes son palabras reservadas en Java para el manejo de excepciones?

Seleccione una:

- a. try, catch, finally, throw, throws ✓
- b. final, static, catch, throws
- c. throw & throws
- d. try, catch, final, throw, throws
- e. Error, Exception, RuntimeException, Throwable

**Justificación:** Las palabras reservadas para el manejo de excepciones son try, catch, finally, throw y throws. La opción d cambia finally por final, que es un modificador sin relación con las excepciones. La c está incompleta, y la e lista nombres de clases, no palabras reservadas.

Fuente: Simulación APX V1 #31 · también aparece en: Java.pdf #163, Java.pdf #35

La respuesta correcta es: try, catch, finally, throw, throws

| **Mi justificación** |
| --- |
| La opción a es la correcta.<br>De las restantes, final es para variables no modificables, y las de la opción e) son conceptos del tema pero no son palabras código para el manejo de excepciones. |

**Pregunta 4**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Para qué sirve la palabra clave throw en Java?

Seleccione una:

- a. Declarar excepciones en la firma de un método.
- b. Definir una clase personalizada de excepción.
- c. Capturar excepciones en un bloque try-catch.
- d. Crear y lanzar explícitamente una excepción. ✓

**Justificación:** throw es la sentencia que crea y lanza explícitamente una excepción, por ejemplo throw new IllegalArgumentException(). La opción a es la más tentadora, pero declarar excepciones en la firma del método le corresponde a throws, no a throw.

Fuente: Quiz APX #2

La respuesta correcta es: Crear y lanzar explícitamente una excepción.

| **Mi justificación** |
| --- |
| Throw crea y lanza una excepción, es teoría básica. |

**Pregunta 5**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes afirmaciones sobre la palabra clave throw es verdadera?

Seleccione una:

a. Se utiliza para detectar excepciones generadas por otros métodos.
b. Se utiliza para declarar que un método puede generar una excepción.
c. Se utiliza para definir el código de limpieza que se debe ejecutar.
d. Se utiliza para generar una excepción manualmente. ✓

**Justificación:** throw sirve para lanzar una excepción de forma explícita (throw new X()). La opción b describe throws, en plural, que declara las excepciones en la firma del método. La a corresponde a try/catch y la c a finally.

Fuente: Simulación APX V2 #45 · también aparece en: Java.pdf #156

La respuesta correcta es: Se utiliza para generar una excepción manualmente.

| **Mi justificación** |
| --- |
| Throw lanza una excepción cuando ocurra una condición especificada manualmente por el usuario. |

**Pregunta 6**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

In Java the difference between throws and throw Is:

Seleccione una:

a. Throws throws an exception and throw indicates the type of exception that the method.

b. Throws is used in methods and throw in constructors.

c. Throws indicates the type of exception that the method does not handle and throw an exception.

**Justificación:** throws se declara en la firma de un método o constructor para indicar qué excepciones no maneja y propaga a quien lo llama. throw es la sentencia que lanza de verdad una excepción. La opción a invierte los papeles, y la b es falsa porque ambos se pueden usar tanto en métodos como en constructores.

Fuente: Java E #5 · también aparece en: Quiz Desarrollo APX 1 #3, Java D #9, Java Sesiones #50

La respuesta correcta es: Throws indicates the type of exception that the method does not handle and throw an exception.

| **Mi justificación** |
| --- |
| Throws indica las excepciones checked que maneja un método, throw lanza una excepción. |

**Pregunta 7**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which of the following statements correctly identifies checked and unchecked exceptions in Java?

Seleccione una:

a. SQLException is an unchecked exception, and ArrayIndexOutOfBoundsException is a checked exception.

b. IOException is a checked exception, and NullPointerException is an unchecked exception. ✓

c. RuntimeException is a checked exception, and FileNotFoundException is an unchecked exception.

d. ArithmeticException is a checked exception, and ClassNotFoundException is an unchecked exception.

**Justificación:** IOException hereda de Exception sin pasar por RuntimeException, así que es checked. NullPointerException hereda de RuntimeException, así que es unchecked. Las demás opciones invierten la clasificación: por ejemplo, RuntimeException es unchecked y FileNotFoundException es checked.

Fuente: Java.pdf #158

La respuesta correcta es: IOException is a checked exception, and NullPointerException is an unchecked exception.

| **Mi justificación** |
| --- |
| IOException es checked porque debemos aclarar que hacer si el archivo a utilizar no existe.<br>NullPointerException es unchecked porque no es necesario aclarar nada más. |

**Pregunta 8**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class Test {
    public static void main(String[] args) {
        try {
            mf();
            System.out.print("A");
        } catch (Exception e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
        System.out.print("D");
    }
    static void mf() {
    }
}
```

Seleccione una:

- a. ABCD
- b. ACD ✓
- c. AC
- d. D
- e. compilation fails
- f. BCD

**Justificación:** mf() no lanza ninguna excepción, así que se imprime A y el catch se salta. finally siempre se ejecuta e imprime C, y después la ejecución sigue con D. Compila aunque se capture Exception sin que nada la lance, porque Exception incluye las excepciones unchecked.

Fuente: Java.pdf #78 · también aparece en: Simulación APX V4 #25

La respuesta correcta es: ACD

?

| **Mi justificación** |
| --- |
| El bloque de try ejecuta void mf() que no hace nada pero no hay problema al compilar, luego imprime A.<br>Ya que no hubo error en try, se va directo al finally, el cual imprime C.<br>Finalmente ejecuta lo que está después que es imprimir D.<br>Se imprimió ACD. |

**Pregunta 9**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué ocurrirá si ejecutas el siguiente código?

```
public class TestThrow {
    public static void validarEdad(int edad) throws IllegalArgumentException {
        if (edad < 18) {
            throw new IllegalArgumentException("Edad no válida para votar.");
        }
        System.out.println("Edad válida.");
    }

    public static void main(String[] args) {
        try {
            validarEdad(16);
        } catch (IllegalArgumentException e) {
            System.out.println("Excepción capturada: " + e.getMessage());
        }
    }
}
```

Seleccione una:

a. Se imprime "Edad válida."

b. Lanza una excepción no controlada.

c. Se imprime "Excepción capturada: Edad no válida para votar." ✓

d. No se imprime nada.

**Justificación:** validarEdad(16) lanza IllegalArgumentException antes de imprimir «Edad válida.», y main la captura e imprime «Excepción capturada: Edad no válida para votar.». La opción b falla porque la excepción sí se controla con el try/catch de main.

Fuente: Quiz Desarrollo APX 2 #10

La respuesta correcta es: Se imprime "Excepción capturada: Edad no válida para votar."

?

| **Mi justificación** |
| --- |
| La excepción de IllegalArgumentException es checked y se programó que, si ocurre, se imprime “Excepción capturada: Edad no válida para votar”. Al ejecutar validarEdad(16) lanza la excepción e imprime la línea antes mencionada. |

**Pregunta 10**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which three are bad practices?

Seleccione una o más de una:

- a. Checking for an IOException and ensuring that the program can recover if one occurs.
- b. Checking for ArrayIndexOutOfBoundsException and ensuring that the program can recover if one occurs. ✓
- c. Checking for FileNotFoundException to inform a user that a filename entered is not valid.
- d. Checking for Error and, if necessary, restarting the program to ensure that users are unaware of problems. ✓
- e. Checking for ArrayIndexOutOfBoundsException when iterating through an array to determine when all elements have been visited. ✓

**Justificación:** ArrayIndexOutOfBoundsException es un error de programación (unchecked): se evita validando el índice, no capturándola para «recuperarse» (b) ni usándola para saber cuándo termina un recorrido (e), que es lo que hace la condición del for. Error indica fallos graves de la JVM (OutOfMemoryError, StackOverflowError) que la aplicación no debe capturar ni ocultar al usuario (d). En cambio, IOException y FileNotFoundException son checked y representan situaciones esperables del entorno: recuperarse o avisar al usuario es justo lo correcto (a y c).

Fuente: Diagnóstico APX 5 #63 · también aparece en: Examen Diagnóstico APX 4 (PDF), pág. 12

Las respuestas correctas son: Checking for ArrayIndexOutOfBoundsException and ensuring that the program can recover if one occurs., Checking for Error and, if necessary, restarting the program to ensure that users are unaware of problems., Checking for ArrayIndexOutOfBoundsException when iterating through an array to determine when all elements have been visited.

| **Mi justificación** |
| --- |
| XXX A) es inválido ya que IOException es checked así que debe programarse la recuperación.<br>VVV B) es válido ya que ArrayOutOfBoundsException es checked así que debe programarse la recuperación.<br>XXX C) es inválido ya que FileNotFound Exception así que debe programarse el mensaje de retorno.<br>VVV D) es válido ya que los Error son unchecked así que no deben programarse.<br>VVV E) es válido ya que es mejor usar collection.length y un ciclo para recorrer los elementos de una colección. |

**Pregunta 11**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles de las siguientes excepciones deben ser chequeadas?

Seleccione una o más de una:

- [x] a. IOException
- [ ] b. IndexOutOfBoundsException
- [x] c. InterruptedException
- [ ] d. NullPointerException
- [ ] e. ClassCastException
- [ ] f. ArithmeticException

**Justificación:** IOException e InterruptedException heredan de Exception pero no de RuntimeException, así que son checked: hay que capturarlas o declararlas. IndexOutOfBoundsException, NullPointerException, ClassCastException y ArithmeticException son subclases de RuntimeException, es decir, unchecked.

Fuente: Simulación APX V1 #5 · también aparece en: Java.pdf #49

Las respuestas correctas son: IOException, InterruptedException

| **Mi justificación** |
| --- |
| Las checked son IOException e InterruptedException, es teoría básica. |

**Pregunta 12**

Parcialmente correcta

Se puntúa 0,50 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which of the following are unchecked exceptions?

Seleccione una o más de una:

- a. ArrayIndexOutOfBoundsException ✓
- b. IOException
- c. RuntimeException ✓
- d. NumberFormatException
- e. IllegalArgumentException ✓
- f. InterruptedException ✗

Ha seleccionado correctamente 3.

**Justificación:** Las excepciones unchecked son RuntimeException y sus subclases, además de Error. Aquí son ArrayIndexOutOfBoundsException, RuntimeException, NumberFormatException (subclase de IllegalArgumentException) e IllegalArgumentException. IOException e InterruptedException son checked. InterruptedException engaña porque aparece mucho con hilos, pero hay que capturarla o declararla.

Fuente: Java.pdf #183

Las respuestas correctas son: ArrayIndexOutOfBoundsException, RuntimeException, NumberFormatException, IllegalArgumentException

| **Mi justificación** |
| --- |
| Las correctas unchecked son la a, c, e y f. Es teoría básica. |

**Pregunta 13**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál será la salida del siguiente fragmento de código?

```
public class TestTryCatch {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println("Resultado: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: División por cero.");
        } catch (Exception e) {
            System.out.println("Error genérico.");
        } finally {
            System.out.println("Ejecución finalizada.");
        }
    }
}
```

Seleccione una o más de una:

- a. Error: División por cero. ✓
- b. Error genérico.
- c. Ejecución finalizada. ✓
- d. Resultado: 1

**Justificación:** 10 / 0 con enteros lanza ArithmeticException, que atrapa el primer catch e imprime «Error: División por cero.». Después se ejecuta siempre el finally, que imprime «Ejecución finalizada.». «Error genérico.» no aparece porque solo se ejecuta el primer catch compatible, y la línea «Resultado» nunca se alcanza.

Fuente: Quiz Desarrollo APX 2 #9

Las respuestas correctas son: Error: División por cero., Ejecución finalizada.

| **Mi justificación** |
| --- |
| La excepción ArithmeticException sale y se ejecuta su catch que imprime “División por cero”. Ya se ejecutó el catch así que nos saltamos al finally, que imprime “Ejecución finalizada”. |

**Pregunta 14**

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál es la salida del siguiente código?

```
public class Test {
    public void run() {
        try {
            System.out.print("Running ");
            throw new Exception();
        } catch (Exception e) {
            System.err.print("Problem");
        } finally {
            System.out.print("Stopping");
        }
    }
    public static void main(String[] args) {
        new Test().run();
    }
}
```

Seleccione una:

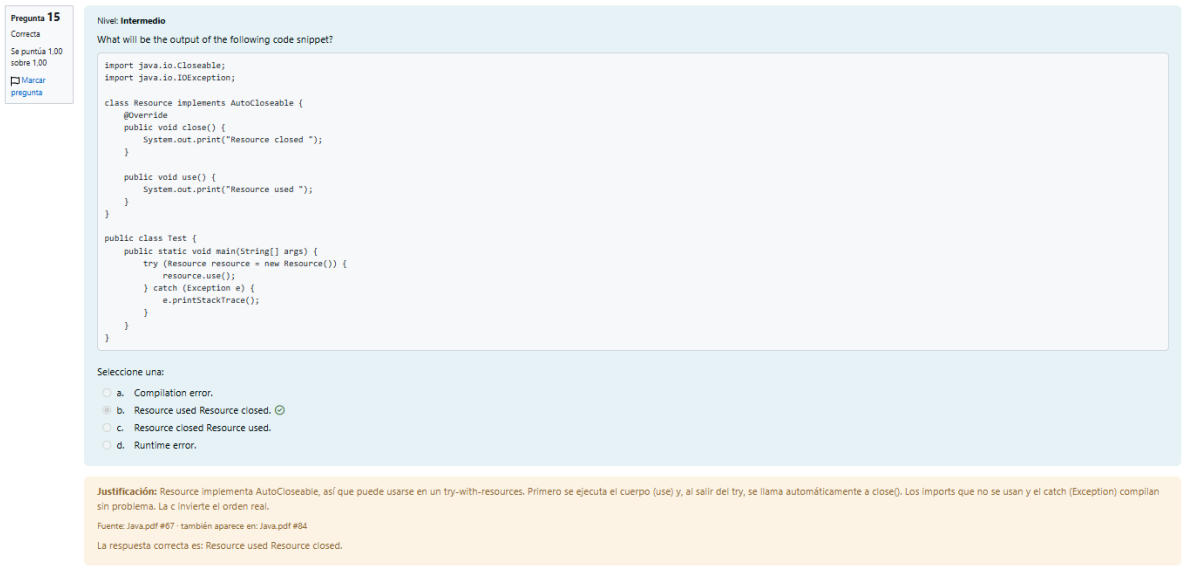
- a. Running Stopping Problem
- b. Running Problem Stopping
- c. Running Problem
- d. La compilación falla
- e. Problem Stopping ⊗
- f. Stopping Problem

**Justificación:** Se imprime "Running ", se lanza la excepción, el catch la atrapa e imprime "Problem", y al final siempre se ejecuta finally, que imprime "Stopping". El orden es try, catch, finally. La opción a invierte catch y finally, y la c olvida que finally siempre se ejecuta. Ojo: "Problem" se escribe en System.err, así que en otras consolas el entrelazado podría variar, pero el orden de ejecución es este.

Fuente: Java.pdf #117

La respuesta correcta es: Running Problem Stopping

| **Mi justificación** |
| --- |
| Bastante directo, se ejecuta el bloque try pero lanza directamente la Exception, entonces se ejecuta el bloque catch e imprime “Problem” y después el bloque finally e imprime “Stopping”.<br><br>Nota: Vi mal y pensé que la impresión de “Running” estaba después de lanzar la excepción. |



**Pregunta 15**

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What will be the output of the following code snippet?

```
import java.io.Closeable;
import java.io.IOException;

class Resource implements AutoCloseable {
    @Override
    public void close() {
        System.out.print("Resource closed ");
    }

    public void use() {
        System.out.print("Resource used ");
    }
}

public class Test {
    public static void main(String[] args) {
        try (Resource resource = new Resource()) {
            resource.use();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Seleccione una:

a. Compilation error.
b. Resource used Resource closed.
c. Resource closed Resource used.
d. Runtime error.

**Justificación:** Resource implementa AutoCloseable, así que puede usarse en un try-with-resources. Primero se ejecuta el cuerpo (use) y, al salir del try, se llama automáticamente a close(). Los imports que no se usan y el catch (Exception) compilan sin problema. La c invierte el orden real.

Fuente: Java.pdf #67 · también aparece en: Java.pdf #84

La respuesta correcta es: Resource used Resource closed.

| **Mi justificación** |
| --- |
| La clase Resource implementa la interfaz AutoClosable así que ya se puede crear una instancia de esta clase dentro de los parámetros de try, entonces se ejecuta use() e imprime “Resource used” y luego ejecuta el método implementado close() e imprime “Resource closed”. |

**Pregunta 16**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which two possible outputs?

```
public class Main {
    public static void main(String[] args) throws Exception {
        doSomething();
    }
    private static void doSomething() throws Exception {
        System.out.println("Before if clause");
        if (Math.random() > 0.5) { throw new Exception();}
        System.out.println("After if clause");
    }
}
```

Seleccione una o más de una:

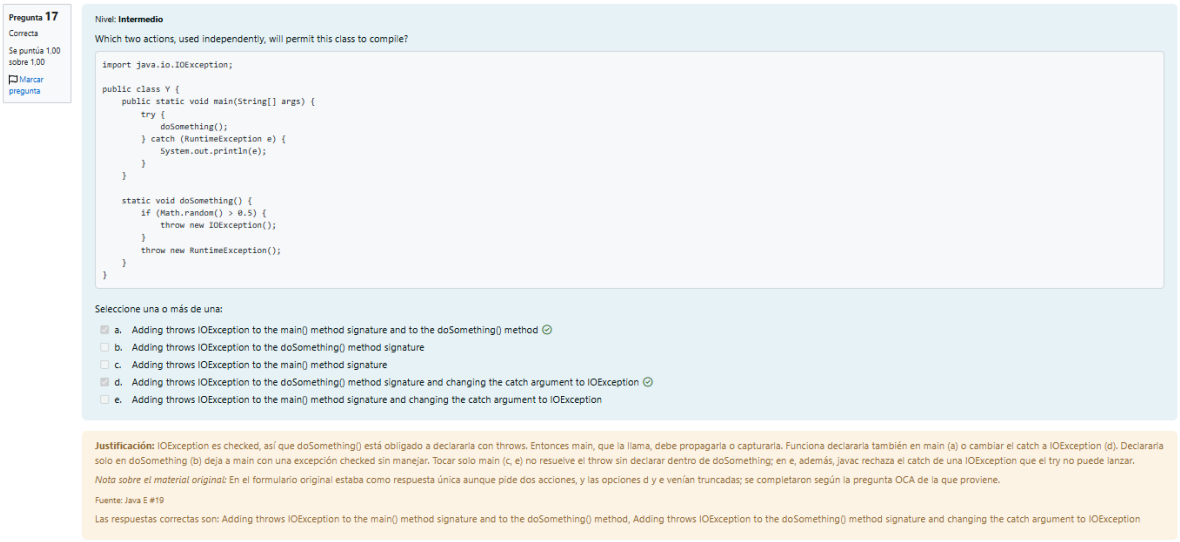
- a. Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15). ✓
- b. Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15) After if clause.
- c. Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15).
- d. Before if clause After if clause. ✓

**Justificación:** Siempre se imprime primero "Before if clause". Después depende de Math.random(). Si se lanza la excepción, sale por main con su stack trace y ya no se imprime "After if clause". Si no se lanza, se imprime "After if clause". La b supone que la ejecución sigue después de la excepción, y la c omite la primera línea.

Fuente: Java Sesiones #27

Las respuestas correctas son: Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15)., Before if clause After if clause.

| **Mi justificación** |
| --- |
| Ya que math.random puede hacer que se lance la excepción causada por Math.random() > 0.5, hay dos casos.<br>Caso 1: Se lanza la excepción.<br>Se imprime “Before if clause” -> Se lanza la excepción<br>Caso 2: No se lanza la excepción.<br>Se imprime “Before if clause” -> No hay excepción -> Se imprime “After if clause” |



**Pregunta 17**

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

Which two actions, used independently, will permit this class to compile?

```
import java.io.IOException;

public class Y {
    public static void main(String[] args) {
        try {
            doSomething();
        } catch (RuntimeException e) {
            System.out.println(e);
        }
    }

    static void doSomething() {
        if (Math.random() > 0.5) {
            throw new IOException();
        }
        throw new RuntimeException();
    }
}
```

Seleccione una o más de una:

- a. Adding throws IOException to the main() method signature and to the doSomething() method ✓
- b. Adding throws IOException to the doSomething() method signature
- c. Adding throws IOException to the main() method signature
- d. Adding throws IOException to the doSomething() method signature and changing the catch argument to IOException ✓
- e. Adding throws IOException to the main() method signature and changing the catch argument to IOException

**Justificación:** IOException es checked, así que doSomething() está obligado a declararla con throws. Entonces main, que la llama, debe propagarla o capturarla. Funciona declararla también en main (a) o cambiar el catch a IOException (d). Declararla solo en doSomething (b) deja a main con una excepción checked sin manejar. Tocar solo main (c, e) no resuelve el throw sin declarar dentro de doSomething; en e, además, javac rechaza el catch de una IOException que el try no puede lanzar.

*Nota sobre el material original:* En el formulario original estaba como respuesta única aunque pide dos acciones, y las opciones d y e venían truncadas; se completaron según la pregunta OCA de la que proviene.

Fuente: Java E #19

Las respuestas correctas son: Adding throws IOException to the main() method signature and to the doSomething() method, Adding throws IOException to the doSomething() method signature and changing the catch argument to IOException

| **Mi justificación** |
| --- |
| Aquí depende de si Math.randon() > 0.5 se cumple o no.<br><br>Si no se cumple no hay que hacer nada ya que RunTimeException ya lo sabe gestionar main().<br><br>Si se cumple, necesitaríamos añadir soporte de IOException al método main y al método doSomething(). O bien podemos cambiar el catch(RunTimeException) por catch(IOException) para que lo sepa gestionar main().<br><br>Son la a) y la d). |

**Pregunta 18**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles son tres malas prácticas?

Seleccione una o más de una:

- a. Comprobación de ArrayIndexOutOfBoundsException al iterar a través de una matriz para determinar cuándo se han visitado todos los elementos.
- b. Verificar errores y, si es necesario, reiniciar el programa para garantizar que los usuarios no se den cuenta de los problemas.
- c. Verificar FileNotFoundException para informar a un usuario que un nombre de archivo ingresado no es válido.
- d. Comprobar si hay ArrayIndexOutOfBoundsException y garantizar que el programa pueda recuperarse si ocurre alguna.
- e. Comprobar si hay una IOException y garantizar que el programa pueda recuperarse si ocurre alguna.

**Justificación:** ArrayIndexOutOfBoundsException es un error de programación (unchecked). No debe usarse para detectar el fin de un recorrido (a) ni para intentar «recuperarse» (d): lo correcto es evitarla respetando los límites del bucle. Ocultar los errores reiniciando el programa (b) también es mala práctica. En cambio, capturar FileNotFoundException para avisar al usuario (c) o IOException para recuperarse (e) son usos correctos de las excepciones checked.

Fuente: Quiz Desarrollo APX 1 #12

Las respuestas correctas son: Comprobación de ArrayIndexOutOfBoundsException al iterar a través de una matriz para determinar cuándo se han visitado todos los elementos., Verificar errores y, si es necesario, reiniciar el programa para garantizar que los usuarios no se den cuenta de los problemas., Comprobar si hay ArrayIndexOutOfBoundsException y garantizar que el programa pueda recuperarse si ocurre alguna.

| **Mi justificación** |
| --- |
| VVV a) Es mala práctica porque es mejor usar array.length y un ciclo for o while.<br>VVV b) Es mala práctica porque los usuarios deben darse cuenta de los problemas-<br>XXX c) Es buena práctica ya que FileNotFoundException es checked así que debe mostrar algún mensaje de error para el usuario.<br>VVV d) Es mala práctica porque ArrayIndexOutOfBoundsException es unchecked así que no debe gestionarse si ocurre.<br>XXX e) Es buena práctica ya que IOException es checked así que sí debe gestionarse si ocurre. |

**Pregunta 19**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Que muestra en pantalla al ejecutar este código?

```
public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }
    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
}
```

Seleccione una:

a. A java.lang.ArrayIndexOutOfBoundsException is thrown.
b. A java.lang.IllegalStateException is thrown.
c. A java.lang.StackOverflowError is thrown. ✓
d. The program runs with no output.
e. Compilation fails

**Justificación:** El índice x % 5 siempre queda entre 0 y 4, así que nunca hay ArrayIndexOutOfBoundsException. Pero go(x) se llama a sí mismo sin caso base, y la recursión infinita agota la pila hasta provocar un StackOverflowError. El distractor tentador es ArrayIndexOutOfBounds, que el módulo 5 evita.

Fuente: Simulación APX V4 #1 · también aparece en: Simulación APX V1 #29

La respuesta correcta es: A java.lang.StackOverflowError is thrown.

| **Mi justificación** |
| --- |
| Al checar array[x%5] se checarán los valores con índice 0,1,2,3 y 4 en bucle indefinidamente, así que lanza un StackOverflowError. |

**Pregunta 20**

Parcialmente correcta

Se puntúa 0,33 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

Which three implementations are valid?

```
interface SampleCloseable {
    public void close() throws java.io.IOException;
}
```

Seleccione una o más de una:

- a. class Test implements SampleCloseable { public void close() throws java.io.IOException { // do something } } ✓
- b. class Test implements SampleCloseable { public void close() throws Exception { // do something } } ✗
- c. class Test implements SampleCloseable { public void close() throws FileNotFoundException { // do something } }
- d. class Test extends SampleCloseable { public void close() throws java.io.IOException { // do something } }
- e. class Test implements SampleCloseable { public void close() { // do something }} ✓

Ha seleccionado correctamente 2.

**Justificación:** Al implementar un método que declara IOException, la implementación puede declarar la misma excepción checked (a), una subclase suya como FileNotFoundException (c) o ninguna (e). No puede declarar una más amplia, como Exception (b). Además, una clase implementa una interfaz con implements, no con extends (d).

Fuente: Java Sesiones #29

Las respuestas correctas son: class Test implements SampleCloseable { public void close() throws java.io.IOException { // do something } }, class Test implements SampleCloseable { public void close() throws FileNotFoundException { // do something } }, class Test implements SampleCloseable { public void close() { // do something }}

| **Mi justificación** |
| --- |
| XXX d) es inválida porque las interfaces en una clase se implementan, no se extienden.<br>XXX c) es inválida porque debe la el método close() heredado debe tener IOException, un padre de este o nada.<br>Las demás son correctas. |

**Pregunta 21**

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
class Exc1 extends Exception {}
class Exc2 extends Exception {}

public class Test {
    static void a() throws Exc1 {
        try {
            System.out.print("Try ");
        } catch (Exc1 e) {
            System.out.print("Catch ");
        } finally {
            System.out.println("Finally ");
        }
    }

    static void b() throws Exc1, Exc2 {
        throw new Exc2();
    }

    public static void main(String args[]) {
        try {
            a();
            b();
        } catch (Exc1 e) {
            System.out.println("Caught1");
        } catch (Exc2 ex) {
            System.out.print("Caught2");
        }
    }
}
```

Seleccione una:

- a. Try Finally Caught2 ✗
- b. La compilación falla
- c. Try
- d. Try Caught2
- e. Try finally

**Justificación:** Exc1 es una excepción checked, y el try dentro de a() solo imprime, así que nunca puede lanzarla. Java prohíbe capturar una excepción checked que el bloque try no puede lanzar, y por eso la compilación falla. La opción a sería la salida si ese catch no existiera, y es la trampa principal.

Fuente: Java.pdf #64 · también aparece en: Simulación APX V4 #24

La respuesta correcta es: La compilación falla

| **Mi justificación** |
| --- |
| Es bastante directo. El método a() imprime el “try”, como no hubo excepción imprime el “finally”. Luego b() lanza la excepción Exc2 la cual es capturada por el catch e imprime “Caught2”. |

**Pregunta 22**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What is the result if you try to compile Truthy.java and then run it with assertions enabled?

```
public class Truthy {
    public static void main(String[] args) {
        int x=7;
        assert (x==6)?"x==6":"x!=6";
    }
}
```

Seleccione una:

- a. Truthy.java does NOT compile.
- b. Truthy.java compiles and the output is x != 6
- c. Truthy.java compiles and an AssertionError is thrown with no additional output
- d. Truthy.java compiles and an AssertionError is thrown with x != 6 as additional output

**Justificación:** La primera expresión de assert tiene que ser boolean. Aquí es un operador ternario cuyo resultado es un String ("x==6" o "x!=6"), así que no compila. La opción d es la tentadora: sería correcta si el código fuera 'assert (x==6) : "x!=6";', con dos puntos separando la condición del mensaje.

Fuente: Java E #2 · también aparece en: Java DE #11

La respuesta correcta es: Truthy.java does NOT compile.

| **Mi justificación** |
| --- |
| Los assert deben tener true o false, y poner el operador ternario : con strings como posibles valores de retorno hace que no sea compatible, así que no compila. |

# RESULTADO = 18/22 = 8.2